



SC3000 Assignment 2

Name	Contribution	Matric Number
Lim Zu Liang	Explanation	U2322044H
Zhao Qixian	Qn1, Qn2 Method2	U2321752L
Mehta Rishika	Qn1, Qn2 Method1	U2323133H

Tutorial Group: SCSD

Teaching Assistant: ZHANG SIYUE

College of Computing and Data Science

Nanyang Technological University

Question 1

sumsum, a competitor of appy, developed some nice smart phone technology called galacticas3, all of which was stolen by stevey, who is a boss of appy. It is unethical for a boss to steal business from rival companies. A competitor is a rival. Smart phone technology is business.

1. Translate the natural language statements above describing the dealing within the Smartphone industry into First Order Logic (FOL).

Statement: sumsum, a competitor of appy

FOL: competitor(sumsum, appy)

Statement: sumsum developed some nice smart phone technology called galactica-s3

FOL: developed_smart_phone(sumsum, galactica_s3)

Statement: galactica-s3 was stolen by stevey

FOL: steal(stevey, galactica_s3)

Statement: stevey, who is a boss of appy

FOL: boss(stevey, appy)

Statement: It is unethical for a boss to steal business from rival companies

FOL: $\forall X \forall C \forall Y \forall T ((\text{boss}(X, C) \wedge \text{steal}(X, T) \wedge \text{developed_smart_phone}(Y, T) \wedge \text{competitor}(Y, C) \wedge \text{business}(T)) \rightarrow \text{unethical}(X))$

Statement: A competitor is a rival

FOL: $\forall C \forall Y (\text{competitor}(Y, C) \rightarrow \text{rival}(Y, C))$

Statement: Smartphone technology is a business

FOL: business(smart_phone_technology)

2. Write these FOL statements as Prolog clauses.

```
% Facts
competitor(sumsum, appy).
developed_smart_phone(sumsum, galactica_s3).
steal(stevey, galactica_s3).
boss(stevey, appy).
smart_phone_technology(galactica_s3).

% Rules
business(T) :-
    smart_phone_technology(T).

unethical(X) :-
    boss(X, C),
    steal(X, T),
    developed_smart_phone(Y, T),
    competitor(Y, C),
    business(T).

rival(Y, C) :-
    competitor(Y, C).
```

3. Using Prolog, prove that Stevey is unethical. Show a trace of your proof.

```
?- unethical(stevey)
|
true.

?- trace, unethical(stevey).
  Call: (13) unethical(stevey) ? creep
  Call: (14) boss(stevey, _25862) ? creep
  Exit: (14) boss(stevey, appy) ? creep
  Call: (14) steal(stevey, _27668) ? creep
  Exit: (14) steal(stevey, galactica_s3) ? creep
  Call: (14) developed_smart_phone(_29474, galactica_s3) ? creep
  Exit: (14) developed_smart_phone(sumsum, galactica_s3) ? creep
  Call: (14) competitor(sumsum, appy) ? creep
  Exit: (14) competitor(sumsum, appy) ? creep
  Call: (14) business(galactica_s3) ? creep
  Call: (15) smart_phone_technology(galactica_s3) ? creep
  Exit: (15) smart_phone_technology(galactica_s3) ? creep
  Exit: (14) business(galactica_s3) ? creep
  Exit: (13) unethical(stevey) ? creep
true.
```

Question 2

The old Royal succession rule states that the throne is passed down along the male line according to the order of birth before the consideration along the female line – similarly according to the order of birth. queen elizabeth, the monarch of United Kingdom, has four offsprings; namely:- prince charles, princess ann, prince andrew and prince edward – listed in the order of birth.

Method 1: born_before(name1,name2)

1. Define their relations and rules in a Prolog rule base. Hence, define the old Royal succession rule. Using this old succession rule determine the line of succession based on the information given. Do a trace to show your results.

```
% Defining parent-child relationships
parent(queen_elizabeth, prince_charles).
parent(queen_elizabeth, princess_ann).
parent(queen_elizabeth, prince_andrew).
parent(queen_elizabeth, prince_edward).

% Defining gender
male(prince_charles).
male(prince_andrew).
male(prince_edward).
female(princess_ann).

% Defining birth order
born_before(prince_charles, princess_ann).
born_before(princess_ann, prince_andrew).
born_before(prince_andrew, prince_edward).

% Old succession rule: males inherit first, then females
old_succession(X) :-
    parent(queen_elizabeth, X),
    male(X).

old_succession(X) :-
    parent(queen_elizabeth, X),
    female(X).

% Ensure order within male and female categories
succession_order(X, Y) :-
    old_succession(X),
    old_succession(Y),
    born_before(X, Y).
```

```
?- consult('/Users/rishi/Desktop/y2s2/SC3000/Sample Prolog Code/qn_2_1.pl').
true.
```

```
?- trace, findall(X, old_succession(X), SuccessionList).
```

```
^ Call: (13) findall(_11862, old_succession(_11862), _11870) ? creep
  Call: (17) old_succession(_11862) ? creep
  Call: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_charles) ? creep
  Call: (18) male(prince_charles) ? creep
  Exit: (18) male(prince_charles) ? creep
  Exit: (17) old_succession(prince_charles) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, princess_ann) ? creep
  Call: (18) male(princess_ann) ? creep
  Fail: (18) male(princess_ann) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_andrew) ? creep
  Call: (18) male(prince_andrew) ? creep
  Exit: (18) male(prince_andrew) ? creep
  Exit: (17) old_succession(prince_andrew) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_edward) ? creep
  Call: (18) male(prince_edward) ? creep
  Exit: (18) male(prince_edward) ? creep
  Exit: (17) old_succession(prince_edward) ? creep
  Redo: (17) old_succession(_11862) ? creep
  Call: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_charles) ? creep
  Call: (18) female(prince_charles) ? creep
  Fail: (18) female(prince_charles) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, princess_ann) ? creep
  Call: (18) female(princess_ann) ? creep
  Exit: (18) female(princess_ann) ? creep
  Exit: (17) old_succession(princess_ann) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_andrew) ? creep
  Call: (18) female(prince_andrew) ? creep
  Fail: (18) female(prince_andrew) ? creep
  Redo: (18) parent(queen_elizabeth, _11862) ? creep
  Exit: (18) parent(queen_elizabeth, prince_edward) ? creep
  Call: (18) female(prince_edward) ? creep
  Fail: (18) female(prince_edward) ? creep
  Fail: (17) old_succession(_11862) ? creep
^ Exit: (13) findall(_11862, user:old_succession(_11862), [prince_charles, prince_andrew, prince_edward, princess_ann]) ? creep
SuccessionList = [prince_charles, prince_andrew, prince_edward, princess_ann].
```

2. Recently, the Royal succession rule has been modified. The throne is now passed down according to the order of birth irrespective of gender. Modify your rules and Prolog knowledge base to handle the new succession rule. Explain the necessary changes to the knowledge needed to represent the new information. Use this new succession rule to determine the new line of succession based on the same knowledge given. Show your results using a trace.

Explanation:

Firstly, we remove the previous 2 **old_succession** predicates (**male(X)** and **female(X)**) because gender-based prioritization is not needed anymore.

Next, we made a new predicate (**new_succession(X)**) to include all children of Queen Elizabeth (using **parent** relationship) regardless of gender.

Lastly, we update the succession ordering rule to **new_succession_order(X, Y)** based purely on the **born_before(X, Y)** relationship.

```
% Defining parent-child relationships
parent(queen_elizabeth, prince_charles).
parent(queen_elizabeth, princess_ann).
parent(queen_elizabeth, prince_andrew).
parent(queen_elizabeth, prince_edward).

% Defining gender
male(prince_charles).
male(prince_andrew).
male(prince_edward).
female(princess_ann).

% Defining birth order
born_before(prince_charles, princess_ann).
born_before(princess_ann, prince_andrew).
born_before(prince_andrew, prince_edward).

% Old succession rule: males inherit first, then females
old_succession(X) :-
parent(queen_elizabeth, X),
male(X).

old_succession(X) :-
parent(queen_elizabeth, X),
female(X),
\+ old_succession(_). % Ensures all males are considered first

% Define order based on birth within the gender category
succession_order(X, Y) :-
old_succession(X),
old_succession(Y),
```

```
born_before(X, Y).
```

```
% New succession rule: direct birth order
```

```
new_succession(X) :-
```

```
parent(queen_elizabeth, X).
```

```
% Define order based purely on birth order
```

```
new_succession_order(X, Y) :-
```

```
new_succession(X),
```

```
new_succession(Y),
```

```
born_before(X, Y).
```

```
?- consult('/Users/rishi/Desktop/y2s2/SC3000/Sample Prolog Code/qn_2_2.pl').  
true.
```

```
?- trace, findall(X, new_succession(X), SuccessionList).
```

```
^ Call: (13) findall(_11862, new_succession(_11862), _11870) ? creep
```

```
Call: (17) new_succession(_11862) ? creep
```

```
Call: (18) parent(queen_elizabeth, _11862) ? creep
```

```
Exit: (18) parent(queen_elizabeth, prince_charles) ? creep
```

```
Exit: (17) new_succession(prince_charles) ? creep
```

```
Redo: (18) parent(queen_elizabeth, _11862) ? creep
```

```
Exit: (18) parent(queen_elizabeth, princess_ann) ? creep
```

```
Exit: (17) new_succession(princess_ann) ? creep
```

```
Redo: (18) parent(queen_elizabeth, _11862) ? creep
```

```
Exit: (18) parent(queen_elizabeth, prince_andrew) ? creep
```

```
Exit: (17) new_succession(prince_andrew) ? creep
```

```
Redo: (18) parent(queen_elizabeth, _11862) ? creep
```

```
Exit: (18) parent(queen_elizabeth, prince_edward) ? creep
```

```
Exit: (17) new_succession(prince_edward) ? creep
```

```
^ Exit: (13) findall(_11862, user:new_succession(_11862), [prince_charles, princess_ann, prince_andrew, prince_edward]) ? creep
```

```
SuccessionList = [prince_charles, princess_ann, prince_andrew, prince_edward].
```

Method 2: birth_order(a, index) and decease simulation

In this method, we have created dead() rule to declare that prince or princess is deceased. This serves as a simulation, where we can state a fact e.g. dead(prince_andrew) and run the succession line again, and prince_andrew will be excluded from the list. In addition, we give index to each newborn prince or princess, instead of the born_before(a,b) rule in method 1, for systematic sort.

```
% Declare dead/1 as dynamic so that it exists even if no clauses are defined.
:- dynamic dead/1.

% Facts: Parent-child relationships
parent(queen_elizabeth, prince_charles).
parent(queen_elizabeth, princess_ann).
parent(queen_elizabeth, prince_andrew).
parent(queen_elizabeth, prince_edward).

% Facts: Gender definitions
male(prince_charles).
male(prince_andrew).
male(prince_edward).
female(princess_ann).

% Facts: Numeric birth order
% Lower numbers indicate earlier birth.
birth_order(prince_charles, 1).
birth_order(princess_ann, 2).
birth_order(prince_andrew, 3).
birth_order(prince_edward, 4).

% Life Status: A child is alive if not marked as dead.
% For example, to declare Prince Andrew as deceased, you would uncomment the line
% below:
% dead(prince_andrew).
% If no fact for dead/1 exists, then \+ dead(X) succeeds.
alive(X) :-
    \+ dead(X).

% Eligible Succession Members:
% Only living children of the specified parent are considered.
succession_member(Parent, Child) :-
    parent(Parent, Child),
    alive(Child).
```

```

% Old Succession Rule:
% Males inherit first (ordered by birth order), then females.
old_succession_rule(Parent) :-
    % Collect all living male heirs with their birth order as Key-Value pairs.
    findall(Order-Child,
        ( succession_member(Parent, Child),
          birth_order(Child, Order),
          male(Child) ),
        MalePairs),
    % Collect all living female heirs with their birth order.
    findall(Order-Child,
        ( succession_member(Parent, Child),
          birth_order(Child, Order),
          female(Child) ),
        FemalePairs),
    % Sort each group by birth order.
    keysort(MalePairs, SortedMalePairs),
    keysort(FemalePairs, SortedFemalePairs),
    % Extract the children from the sorted pairs.
    extract_children(SortedMalePairs, SortedMales),
    extract_children(SortedFemalePairs, SortedFemales),
    % Append the male heirs first, then female heirs.
    append(SortedMales, SortedFemales, Succession),
    format('Old succession order: ~w~n', [Succession]).

% New Succession Rule:
% The throne is passed solely according to the order of birth, irrespective of
gender.
new_succession_rule(Parent) :-
    % Collect all living children with their birth order as Key-Value pairs.
    findall(Order-Child,
        ( succession_member(Parent, Child),
          birth_order(Child, Order) ),
        Pairs),
    % Sort the list by birth order.
    keysort(Pairs, SortedPairs),
    % Extract the children from the sorted pairs.
    extract_children(SortedPairs, Succession),
    format('New succession order: ~w~n', [Succession]).

% Helper predicate: Extract the children (values) from a list of Key-Value pairs.
extract_children([], []).
extract_children([_ -Child|RestPairs], [Child|RestChildren]) :-
    extract_children(RestPairs, RestChildren).

```

To find out the old succession sequence, we can run
 trace,old_succession_rule(queen_elizabeth).

```
% d:/OneDrive/Documents/GitHub/SC3000_AI/Project-2/qa_2_2_method2.pl compiled 0.00 sec, 10 clauses
?- trace,old_succession_rule(queen_elizabeth).
Call: (13) old_succession_rule(queen_elizabeth) ? creep
Exit: (13) old_succession_rule(queen_elizabeth) ? creep
^
Call: (14) findall(_18362=_18364, (succession_member(queen_elizabeth, _18364), birth_order(_18364, _18362), male(_18364))), _18394) ? creep
Call: (15) succession_member(queen_elizabeth, _18364) ? creep
Call: (20) parent(queen_elizabeth, _18364) ? creep
Exit: (20) parent(queen_elizabeth, prince_charles) ? creep
Call: (20) alive(prince_charles) ? creep
Call: (21) dead(prince_charles) ? creep
Fail: (21) dead(prince_charles) ? creep
Redo: (20) alive(prince_charles) ? creep
Exit: (20) alive(prince_charles) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_charles) ? creep
Call: (19) birth_order(prince_charles, _18362) ? creep
Exit: (19) birth_order(prince_charles, 1) ? creep
Call: (19) male(prince_charles) ? creep
Exit: (19) male(prince_charles) ? creep
Redo: (20) parent(queen_elizabeth, _18364) ? creep
Exit: (20) parent(queen_elizabeth, princess_anna) ? creep
Call: (20) alive(princess_anna) ? creep
Call: (21) dead(princess_anna) ? creep
Fail: (21) dead(princess_anna) ? creep
Redo: (20) alive(princess_anna) ? creep
Exit: (20) alive(princess_anna) ? creep
Exit: (19) succession_member(queen_elizabeth, princess_anna) ? creep
Call: (19) birth_order(princess_anna, _18362) ? creep
Exit: (19) birth_order(princess_anna, 2) ? creep
Call: (19) male(princess_anna) ? creep
Fail: (19) male(princess_anna) ? creep
Redo: (20) parent(queen_elizabeth, _18364) ? creep
Exit: (20) parent(queen_elizabeth, prince_andrew) ? creep
Call: (20) alive(prince_andrew) ? creep
Call: (21) dead(prince_andrew) ? creep
Fail: (21) dead(prince_andrew) ? creep
Redo: (20) alive(prince_andrew) ? creep
Exit: (20) alive(prince_andrew) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_andrew) ? creep
Call: (19) birth_order(prince_andrew, _18362) ? creep
Call: (19) birth_order(prince_andrew, 3) ? creep
Call: (19) male(prince_andrew) ? creep
Exit: (19) male(prince_andrew) ? creep
Redo: (20) parent(queen_elizabeth, _18364) ? creep
Exit: (20) parent(queen_elizabeth, prince_edward) ? creep
Call: (20) alive(prince_edward) ? creep
Call: (21) dead(prince_edward) ? creep
Fail: (21) dead(prince_edward) ? creep
Redo: (20) alive(prince_edward) ? creep
Exit: (20) alive(prince_edward) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_edward) ? creep
Call: (19) birth_order(prince_edward, _18362) ? creep
Exit: (19) birth_order(prince_edward, 4) ? creep
Call: (19) male(prince_edward) ? creep
Exit: (19) male(prince_edward) ? creep
Exit: (14) findall(_84=_86, (succession_member(queen_elizabeth, _18364), birth_order(_18364, _18362), male(_18364))), [1-prince_charles, 3-prince_andrew, 4-prince_edward]) ? creep
^
Call: (14) findall(_84=_86, (succession_member(queen_elizabeth, _86), birth_order(_86, _84), female(_86))), [1-prince_charles, 3-prince_andrew, 4-prince_edward]) ? creep
Call: (19) succession_member(queen_elizabeth, _86) ? creep
Call: (20) parent(queen_elizabeth, _86) ? creep
Exit: (20) parent(queen_elizabeth, prince_charles) ? creep
Call: (20) alive(prince_charles) ? creep
Call: (21) dead(prince_charles) ? creep
Fail: (21) dead(prince_charles) ? creep
Redo: (20) alive(prince_charles) ? creep
Exit: (20) alive(prince_charles) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_charles) ? creep
Call: (19) birth_order(prince_charles, _84) ? creep
Exit: (19) birth_order(prince_charles, 1) ? creep
Call: (19) female(prince_charles) ? creep
Fail: (19) female(prince_charles) ? creep
Redo: (20) parent(queen_elizabeth, _86) ? creep
Exit: (20) parent(queen_elizabeth, princess_anna) ? creep
Call: (20) alive(princess_anna) ? creep
Call: (21) dead(princess_anna) ? creep
Fail: (21) dead(princess_anna) ? creep
Redo: (20) alive(princess_anna) ? creep
Exit: (20) alive(princess_anna) ? creep
Exit: (19) succession_member(queen_elizabeth, princess_anna) ? creep
Call: (19) birth_order(princess_anna, _84) ? creep
Exit: (19) birth_order(princess_anna, 2) ? creep
Call: (19) female(princess_anna) ? creep
Exit: (19) female(princess_anna) ? creep
Redo: (20) parent(queen_elizabeth, _86) ? creep
Exit: (20) parent(queen_elizabeth, prince_andrew) ? creep
Call: (20) alive(prince_andrew) ? creep
Call: (21) dead(prince_andrew) ? creep
Fail: (21) dead(prince_andrew) ? creep
Redo: (20) alive(prince_andrew) ? creep
Exit: (20) alive(prince_andrew) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_andrew) ? creep
Call: (19) birth_order(prince_andrew, _84) ? creep
Exit: (19) birth_order(prince_andrew, 3) ? creep
Call: (19) female(prince_andrew) ? creep
Fail: (19) female(prince_andrew) ? creep
Redo: (20) parent(queen_elizabeth, _86) ? creep
Exit: (20) parent(queen_elizabeth, prince_edward) ? creep
Call: (20) alive(prince_edward) ? creep
Call: (21) dead(prince_edward) ? creep
Fail: (21) dead(prince_edward) ? creep
Redo: (20) alive(prince_edward) ? creep
Exit: (20) alive(prince_edward) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_edward) ? creep
Call: (19) birth_order(prince_edward, _84) ? creep
Exit: (19) birth_order(prince_edward, 4) ? creep
Call: (19) female(prince_edward) ? creep
Fail: (19) female(prince_edward) ? creep
Exit: (14) findall(_84=_86, user:(succession_member(queen_elizabeth, _86), birth_order(_86, _84), female(_86))), [2-princess_anna]) ? creep
^
Call: (14) keysort([1-prince_charles, 3-prince_andrew, 4-prince_edward], _46168) ? creep
Exit: (14) keysort([1-prince_charles, 3-prince_andrew, 4-prince_edward], [1-prince_charles, 3-prince_andrew, 4-prince_edward]) ? creep
Call: (14) keysort([2-princess_anna], _47992) ? creep
Exit: (14) keysort([2-princess_anna], [2-princess_anna]) ? creep
Call: (14) extract_children([1-prince_charles, 3-prince_andrew, 4-prince_edward], _49804) ? creep
Call: (15) extract_children([3-prince_andrew, 4-prince_edward], _50712) ? creep
Call: (15) extract_children([4-prince_edward], _51620) ? creep
Call: (17) extract_children([], _52528) ? creep
Exit: (17) extract_children([], []) ? creep
Exit: (16) extract_children([4-prince_edward], [prince_edward]) ? creep
Exit: (15) extract_children([3-prince_andrew, 4-prince_edward], [prince_andrew, prince_edward]) ? creep
Exit: (14) extract_children([1-prince_charles, 3-prince_andrew, 4-prince_edward], [prince_charles, prince_andrew, prince_edward]) ? creep
Call: (14) extract_children([2-princess_anna], _57040) ? creep
Call: (15) extract_children([1-princess_anna], _57948) ? creep
Exit: (15) extract_children([], []) ? creep
Exit: (14) extract_children([2-princess_anna], [princess_anna]) ? creep
Call: (14) lists:append([prince_charles, prince_andrew, prince_edward], [princess_anna], _60656) ? creep
Exit: (14) lists:append([prince_charles, prince_andrew, prince_edward], [princess_anna], [prince_charles, prince_andrew, prince_edward, princess_anna]) ? creep
Call: (14) format('Old succession order: ~v~n', [[prince_charles, prince_andrew, prince_edward, princess_anna]]) ? creep
Old succession order: [prince_charles, prince_andrew, prince_edward, princess_anna]
Exit: (14) format('Old succession order: ~v~n', [[prince_charles, prince_andrew, prince_edward, princess_anna]]) ? creep
Exit: (13) old_succession_rule(queen_elizabeth) ? creep
true.
```

To view the result easily:

```
[debug] ?- old_succession_rule(queen_elizabeth).  
Old succession order: [prince_charles,prince_andrew,prince_edward,princess_ann]  
true.
```

To find out the new succession sequence, we can run
trace,new_succession_rule(queen_elizabeth).

```
[debug] ?- trace,new_succession_rule(queen_elizabeth).  
^ Call: (13) new_succession_rule(queen_elizabeth) ? creep  
Call: (14) findall(_8084-8086, (succession_member(queen_elizabeth, _8086), birth_order(_8086, _8084)), _8106) ? creep  
Exit: (19) succession_member(queen_elizabeth, _8086) ? creep  
Call: (20) parent(queen_elizabeth, _8086) ? creep  
Exit: (20) parent(queen_elizabeth, prince_charles) ? creep  
Call: (20) alive(prince_charles) ? creep  
Call: (21) dead(prince_charles) ? creep  
Fail: (21) dead(prince_charles) ? creep  
Redo: (20) alive(prince_charles) ? creep  
Exit: (20) alive(prince_charles) ? creep  
Exit: (19) succession_member(queen_elizabeth, prince_charles) ? creep  
Call: (19) birth_order(prince_charles, _8084) ? creep  
Exit: (19) birth_order(prince_charles, 1) ? creep  
Redo: (20) parent(queen_elizabeth, _8086) ? creep  
Exit: (20) parent(queen_elizabeth, princess_ann) ? creep  
Call: (20) alive(princess_ann) ? creep  
Call: (21) dead(princess_ann) ? creep  
Fail: (21) dead(princess_ann) ? creep  
Redo: (20) alive(princess_ann) ? creep  
Exit: (20) alive(princess_ann) ? creep  
Exit: (19) succession_member(queen_elizabeth, princess_ann) ? creep  
Call: (19) birth_order(princess_ann, _8084) ? creep  
Exit: (19) birth_order(princess_ann, 2) ? creep  
Redo: (20) parent(queen_elizabeth, _8086) ? creep  
Exit: (20) parent(queen_elizabeth, prince_andrew) ? creep  
Call: (20) alive(prince_andrew) ? creep  
Call: (21) dead(prince_andrew) ? creep  
Fail: (21) dead(prince_andrew) ? creep  
Redo: (20) alive(prince_andrew) ? creep  
Exit: (20) alive(prince_andrew) ? creep  
Exit: (19) succession_member(queen_elizabeth, prince_andrew) ? creep  
Call: (19) birth_order(prince_andrew, _108) ? creep  
Exit: (19) birth_order(prince_andrew, 3) ? creep  
Redo: (20) parent(queen_elizabeth, _110) ? creep  
Exit: (20) parent(queen_elizabeth, prince_edward) ? creep  
Call: (20) alive(prince_edward) ? creep  
Call: (21) dead(prince_edward) ? creep  
Fail: (21) dead(prince_edward) ? creep  
Redo: (20) alive(prince_edward) ? creep  
Exit: (20) alive(prince_edward) ? creep  
Exit: (19) succession_member(queen_elizabeth, prince_edward) ? creep  
Call: (19) birth_order(prince_edward, _108) ? creep  
Exit: (19) birth_order(prince_edward, 4) ? creep  
Call: (14) findall(_108-110, user (succession_member(queen_elizabeth, _110), birth_order(_110, _108)), [1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward]) ? creep  
Call: (14) keysort([1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward], _15500) ? creep  
Exit: (14) keysort([1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward], [1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward]) ? creep  
Call: (14) extract_children([1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward], _17330) ? creep  
Call: (15) extract_children([2-princess_ann, 3-prince_andrew, 4-prince_edward], _18238) ? creep  
Call: (16) extract_children([3-prince_andrew, 4-prince_edward], _19146) ? creep  
Call: (17) extract_children([4-prince_edward], _20054) ? creep  
Call: (18) extract_children([], _20962) ? creep  
Exit: (18) extract_children([], []) ? creep  
Exit: (17) extract_children([4-prince_edward], [prince_edward]) ? creep  
Exit: (16) extract_children([3-prince_andrew, 4-prince_edward], [prince_andrew, prince_edward]) ? creep  
Exit: (15) extract_children([2-princess_ann, 3-prince_andrew, 4-prince_edward], [princess_ann, prince_andrew, prince_edward]) ? creep  
Exit: (14) extract_children([1-prince_charles, 2-princess_ann, 3-prince_andrew, 4-prince_edward], [prince_charles, princess_ann, prince_andrew, prince_edward]) ? creep  
^ Call: (14) forall('New succession order: ~ys', [(prince_charles, princess_ann, prince_andrew, prince_edward)]) ? creep  
New succession order: [prince_charles,princess_ann,prince_andrew,prince_edward]  
Exit: (14) forall('New succession order: ~ys', [(prince_charles, princess_ann, prince_andrew, prince_edward)]) ? creep  
Exit: (13) new_succession_rule(queen_elizabeth) ? creep  
true.
```

To view the result easily:

```
[debug] ?- new_succession_rule(queen_elizabeth).  
New succession order: [prince_charles,princess_ann,prince_andrew,prince_edward]  
true.
```

To simulate the decease of prince or princess, for example prince_andrew. We can either add
dead(prince_andrew) into our knowledge base prior to running, or we can run
assert(dead(prince_andrew)). in the Prolog terminal while running. For demonstration purpose,
we will run assert(dead(prince_andrew)).

```
[debug] ?- assert(dead(prince_andrew)).  
true.
```

```
[debug] ?- new_succession_rule(queen_elizabeth).  
New succession order: [prince_charles,princess_ann,prince_edward]  
true.
```

We can see that prince_andrew has been removed from the succession line

```
[debug] ?- trace, new_succession_rule(queen_elizabeth).
Call: (13) new_succession_rule(queen_elizabeth) ? creep
^ Call: (14) findall(_7710-7712, (succession_member(queen_elizabeth, _7712), birth_order(_7712, _7710)), _7732) ? creep
Call: (19) succession_member(queen_elizabeth, _7712) ? creep
Call: (20) parent(queen_elizabeth, _7712) ? creep
Exit: (20) parent(queen_elizabeth, prince_charles) ? creep
Call: (20) alive(prince_charles) ? creep
Call: (21) dead(prince_charles) ? creep
Fail: (21) dead(prince_charles) ? creep
Redo: (20) alive(prince_charles) ? creep
Exit: (20) alive(prince_charles) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_charles) ? creep
Call: (19) birth_order(prince_charles, _7710) ? creep
Exit: (19) birth_order(prince_charles, 1) ? creep
Redo: (20) parent(queen_elizabeth, _7712) ? creep
Exit: (20) parent(queen_elizabeth, princess_ann) ? creep
Call: (20) alive(princess_ann) ? creep
Call: (21) dead(princess_ann) ? creep
Fail: (21) dead(princess_ann) ? creep
Redo: (20) alive(princess_ann) ? creep
Exit: (20) alive(princess_ann) ? creep
Exit: (19) succession_member(queen_elizabeth, princess_ann) ? creep
Call: (19) birth_order(princess_ann, _7710) ? creep
Exit: (19) birth_order(princess_ann, 2) ? creep
Redo: (20) parent(queen_elizabeth, _7712) ? creep
Exit: (20) parent(queen_elizabeth, prince_andrew) ? creep
Call: (20) alive(prince_andrew) ? creep
Call: (21) dead(prince_andrew) ? creep
Exit: (21) dead(prince_andrew) ? creep
Fail: (20) alive(prince_andrew) ? creep
Redo: (20) parent(queen_elizabeth, _110) ? creep
Exit: (20) parent(queen_elizabeth, prince_edward) ? creep
Call: (20) alive(prince_edward) ? creep
Call: (21) dead(prince_edward) ? creep
Fail: (21) dead(prince_edward) ? creep
Redo: (20) alive(prince_edward) ? creep
Exit: (20) alive(prince_edward) ? creep
Exit: (19) succession_member(queen_elizabeth, prince_edward) ? creep
Call: (19) birth_order(prince_edward, _108) ? creep
Exit: (19) birth_order(prince_edward, 4) ? creep
^ Exit: (14) findall(_108-110, user:(succession_member(queen_elizabeth, _110), birth_order(_110, _108)), [1-prince_charles, 2-princess_ann, 4-prince_edward]) ? creep
Call: (14) keysort([1-prince_charles, 2-princess_ann, 4-prince_edward], _11712) ? creep
Exit: (14) keysort([1-prince_charles, 2-princess_ann, 4-prince_edward], [1-prince_charles, 2-princess_ann, 4-prince_edward]) ? creep
Call: (14) extract_children([1-prince_charles, 2-princess_ann, 4-prince_edward], _13536) ? creep
Call: (15) extract_children([2-princess_ann, 4-prince_edward], _14444) ? creep
Call: (16) extract_children([4-prince_edward], _15352) ? creep
Call: (17) extract_children([], _16260) ? creep
Exit: (17) extract_children([], []) ? creep
Exit: (16) extract_children([4-prince_edward], [prince_edward]) ? creep
Exit: (15) extract_children([2-princess_ann, 4-prince_edward], [princess_ann, prince_edward]) ? creep
Exit: (14) extract_children([1-prince_charles, 2-princess_ann, 4-prince_edward], [prince_charles, princess_ann, prince_edward]) ? creep
^ Call: (14) forall('New succession order: ~v~n', [[prince_charles, princess_ann, prince_edward]]) ? creep
New succession order: [prince_charles, princess_ann, prince_edward]
^ Exit: (14) forall('New succession order: ~v~n', [[prince_charles, princess_ann, prince_edward]]) ? creep
Exit: (13) new_succession_rule(queen_elizabeth) ? creep
true.
```

After asserting `dead(prince_andrew)`, in the trace, when `alive(prince_andrew)` is called, the system asks `dead(prince_andrew)`, which returns true, causing `alive(prince_andrew)` to fail. Consequently, only the living children are in the list [1-prince_charles, 2-princess_ann, 4-prince_edward]. The key sort and subsequent extraction process this reduced list, producing the final succession order [prince_charles, princess_ann, prince_edward]. Thus, Prince Andrew is omitted from the new line of succession.